C#_Day1_Hands_On_Koduru_Ushasree

Level-1 Problem 1: Student Grade Evaluator

Scenario

You are developing a console-based application in .NET 8 for a school. The application should evaluate a student's marks and assign a grade based on predefined rules.

Requirements

- Accept student name and marks (0-100).
- Use if-else statements to determine grade.
- Display grade as A, B, C, D or Fail.
- Handle invalid input using conditional checks.

Technical Constraints

- Use C# (.NET 8 Console Application).
- Use appropriate data types (string, int).
- Use if-else control flow.
- Do not use advanced concepts like classes or LINQ.

Sample Input

Enter Name: Rahul
Enter Marks: 78

Sample Output

Student: Rahul
Grade: B

Expectations

Program should correctly evaluate grade and handle edge cases like marks below 0 or above 100.

Learning Outcome

Understand variables, data types, input/output handling and if-else control statements in C#.

Code :-

```
ConsoleApp1    p4.cs    p3.cs    p2.cs    p1.cs    Program.cs
ConsoleApp1    ConsoleApp1.p1    Main(string[] args)

using System;
using System.Collections.Generic;
using System.Linq;
using System.Reflection.Emit;
using System.Text;
using System.Threading.Tasks;

namespace ConsoleApp1
{
    0 references
    internal class p1
    {
        0 references
        static void Main(string[] args)
        {
            Console.Write("Enter Name: ");
            string name = Console.ReadLine();

            // Validate Name
            if (string.IsNullOrWhiteSpace(name))
            {
                Console.WriteLine("Invalid Name! Please enter a valid name.");
            }
            else
            {
                Console.Write("Enter Marks: ");
                int marks = Convert.ToInt32(Console.ReadLine());
                // Validate Marks
                if (marks < 0 || marks > 100)
                {
                    Console.WriteLine("Invalid Marks! Please enter marks between 0 and 100");
                }
                else
                {
                    string grade;
```

```
                    if (marks >= 90)
                        grade = "A";
                    else if (marks >= 80)
                        grade = "B";
                    else if (marks >= 70)
                        grade = "C";
                    else if (marks >= 60)
                        grade = "D";
                    else
                        grade = "Fail";

                    Console.WriteLine("Student: " + name);
                    Console.WriteLine("Grade: " + grade);
                }
            }
        }
    }
}
```

Output :-

```
Enter Name:
Invalid Name! Please enter a valid name.

C:\Users\ushus\source\repos\ConsoleApp1\bin\Debug\net8.0\ConsoleApp1.exe (process 35436) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

```
Enter Name: Usha
Enter Marks: 90
Student: Usha
Grade: A

C:\Users\ushus\source\repos\ConsoleApp1\bin\Debug\net8.0\ConsoleApp1.exe (process 12892) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

Technical Requirements :-

This program is developed using C# in a .NET 8 Console Application. It takes the student's name and marks as input from the user through the console. The name is stored using the string data type and marks are stored using the int data type. The program uses if-else conditional statements to check whether the marks fall within the valid range and to determine the appropriate grade. If the input is invalid, the program displays an error message; otherwise, it prints the student name along with the calculated grade.

Level-1 Problem 2: Simple Calculator Using Switch

Scenario

Create a simple calculator application that performs basic arithmetic operations.

Requirements

- Accept two numbers from user.
- Accept operator (+, -, *, /).
- Use switch statement to perform operation.
- Display result.

Technical Constraints

- Use int or double data types.
- Use switch-case statement.
- Handle division by zero.

Sample Input

Enter First Number: 10
Enter Second Number: 5
Enter Operator: *

Sample Output

Result: 50

Expectations

Correct operator selection and proper validation of inputs.

Learning Outcome

Understand switch statements, arithmetic operators and control flow in C#.

Code :-

```
namespace ConsoleApp1
{
    0 references
    internal class p2
    {
        0 references
        static void Main(string[] args)
        {
            Console.WriteLine("Enter first number: ");
            double num1 = Convert.ToDouble(Console.ReadLine());

            Console.WriteLine("Enter second number: ");
            double num2 = Convert.ToDouble(Console.ReadLine());

            Console.WriteLine("Enter Operator: (+, -, *, /)");
            string op = Console.ReadLine();

            string result = op switch
            {
                "+" => "Result: " + (num1 + num2),
                "-" => "Result: " + (num1 - num2),
                "*" => "Result: " + (num1 * num2),
                "/" when num2 == 0 => "Cannot Divide By Zero",
                "/" => "Result: " + (num1 / num2),
                _ => "Invalid Operator"

            };
            Console.WriteLine("Result:" + result);
        }
    }
}
```

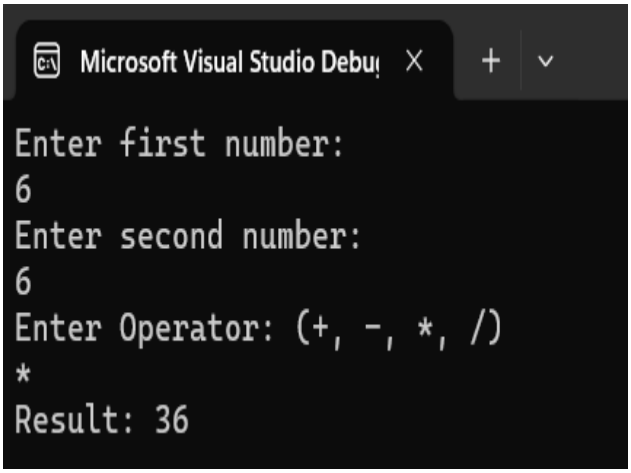
Output :-

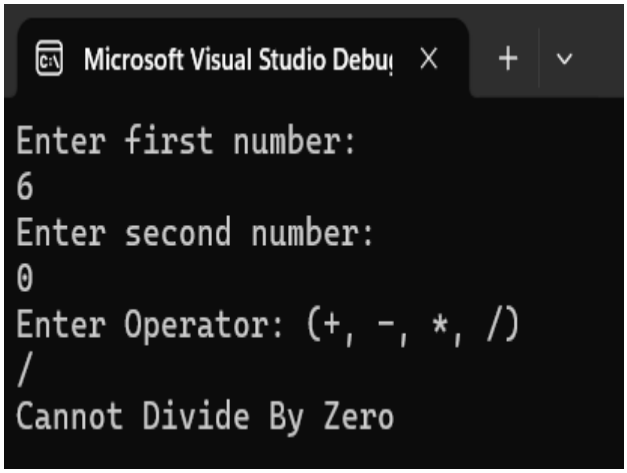
```
Microsoft Visual Studio Debu

Enter first number:
6
Enter second number:
6
Enter Operator: (+, -, *, /)
+
Result: 12
```

```
Microsoft Visual Studio Debu

Enter first number:
6
Enter second number:
6
Enter Operator: (+, -, *, /)
-
Result: 0
```





Technical Requirements :-

This program is implemented using C# in a .NET 8 console application to perform basic arithmetic calculations. It accepts two numeric values from the user and stores them using the double data type. The user is also asked to enter an arithmetic operator such as *+, −,* , or /* to specify the operation. A switch statement is used to determine which calculation should be performed based on the operator entered. The program also includes a condition to prevent division by zero and displays the result or an appropriate error message if the operator is invalid.

Level-2 Problem 1: Employee Bonus Calculator

Scenario

Develop a console application that calculates employee bonus based on salary and years of experience.

Requirements

- Accept employee name, salary and years of experience.
- Use if-else and conditional operator.
- Bonus rules:
  - Experience < 2 years: 5% bonus
  - 2-5 years: 10% bonus
  - >5 years: 15% bonus
- Display final salary after bonus.

Technical Constraints

- Use double for salary.
- Use if-else and ternary operator.
- Use proper formatting for currency output.

Sample Input

Enter Name: Aisha
Enter Salary: 50000
Enter Experience: 4

Sample Output

Employee: Aisha
Bonus: 5000
Final Salary: 55000

Expectations

Accurate bonus calculation and correct usage of control statements.

Learning Outcome

Apply conditional logic and arithmetic operations in real-world scenarios.

Code :-

```
namespace ConsoleApp1
{
    0 references
    internal class p3
    {
        0 references
        static void Main(string[] args)
        {
            Console.WriteLine("Enter Name: ");
            string name = Console.ReadLine();

            if (string.IsNullOrWhiteSpace(name))
            {
                Console.WriteLine("Invalid Name!");
            }

            Console.WriteLine("Enter Salary: ");

            if (!double.TryParse(Console.ReadLine(), out double salary) || salary < 0)
            {
                Console.WriteLine("Invalid salary");
            }

            Console.WriteLine("Enter Years of Experience: ");

            if (!int.TryParse(Console.ReadLine(), out int exp) || exp < 0)
            {
                Console.WriteLine("Invalid Experience");
            }

            double bonusPercent;
            if (exp < 2)
            {
                bonusPercent = 0.05;
```

```
                }
                else if (exp <= 5)
                {
                    bonusPercent = 0.10;

                }
                else
                {
                    bonusPercent = 0.15;
                }
                double bonus = salary * bonusPercent;
                Console.WriteLine("Employee: " + name);
                Console.WriteLine("Bonus: " + bonus);
                Console.WriteLine("Final Salary: " + (salary + bonus));
            }
        }
    }
```

Output :-

```
Microsoft Visual Studio Debu

Enter Name:
Usha
Enter Salary:
50000
Enter Years of Experience:
2
Employee: Usha
Bonus: 5000
Final Salary: 55000
```

```
Microsoft Visual Studio Debu

Enter Name:
Sree
Enter Salary:
60000
Enter Years of Experience:
6
Employee: Sree
Bonus: 9000
Final Salary: 69000
```

Technical Requirements :-

This application is developed using C# in a .NET 8 console environment to calculate an employee's bonus based on their salary and years of experience. The program accepts the employee's name, salary, and experience as input from the user. Appropriate data types such as string, double, and int are used to store these values. Conditional statements like if-else are used to determine the correct bonus percentage according to the experience rules. The program then calculates the bonus amount and displays the employee name, bonus earned, and the final salary after adding the bonus.

Level-2 Problem 2: Number Analysis Using Loops

Scenario

Create a .NET 8 console application that analyzes numbers between 1 and N.

Requirements

- Accept a number N from user.
- Use loops to:
  - Count even numbers
  - Count odd numbers
  - Calculate sum of all numbers
- Display results.

Technical Constraints

- Use for or while loop.
- Use int data type.
- Avoid using arrays or collections.

Sample Input

Enter Number: 10

Sample Output

Even Count: 5
Odd Count: 5
Sum: 55

Expectations

Proper loop usage and correct counting logic.

Learning Outcome

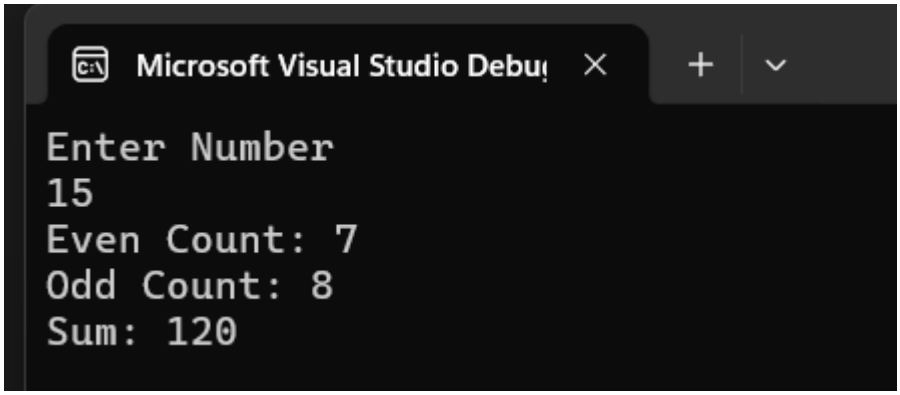
Strengthen understanding of loops, counters and accumulators in C#.

Code :-

```
namespace ConsoleApp1
{
    0 references
    internal class p4
    {
        0 references
        static void Main(string[] args)
        {
            Console.WriteLine("Enter Number");
            if (!int.TryParse(Console.ReadLine(), out int num) || num < 0)
            {
                Console.WriteLine("Invalid Number");
            }
            else
            {
                int evenCount = 0;
                int oddCount = 0;
                int sum = 0;
                for (int i = 1; i <= num; i++)
                {
                    sum += i;
                    if (i % 2 == 0)
                    {
                        evenCount++;
                    }
                    else
                    {
                        oddCount++;
                    }
                }
                Console.WriteLine("Even Count: " + evenCount);
                Console.WriteLine("Odd Count: " + oddCount);
                Console.WriteLine("Sum: " + sum);
            }

        }
```

Output :-



Technical Requirements :-

This program is developed using C# in a .NET 8 console application to analyze numbers from 1 to N. The application accepts a number from the user and validates the input using int.TryParse. A for loop is used to iterate through all numbers from 1 up to the entered value. During the loop execution, counters are used to track the number of even and odd values while another variable accumulates the total sum. Finally, the program displays the count of even numbers, odd numbers, and the sum of all numbers within the range.